

### Exercice 1 :

Ecrivez un programme ptr.cpp qui :

1. déclare un entier n ;
2. déclare une référence ref vers cet entier ;
3. déclare un pointeur ptr vers cet entier ;
4. dans les trois cas, imprimer la valeur de n et son adresse (en utilisant, n, ref et ptr).

**Exercice 2 :** Soit le modèle de structure suivant :

```
struct essai {
    int n;
    float x;
};
```

Ecrire une fonction nommée raz permettant de mettre à zéro les 2 champs d'une structure de ce type transmise en argument :

1. Par adresse.
2. Par référence.

**Exercice 3 :** Ecrire une fonction qui prend en arguments deux chaînes de caractères s1 et s2 et renvoie 1 si la taille de s1 < s2, 0 sinon.

Donner le résultat de programme ci-dessous, puis tester le sur la machine.

```
char &get_val(string &str, int ix)
{ return str[ix]; }
int main(){
    string s("a value");
    cout << s << endl;
    get_val(s, 0) = 'A';
    cout << s << endl
    return 0;}
```

**Exercice 4 :** La fonction f() effectue la transmission par référence de sa valeur de retour. Les exemples ci-dessous sont-ils corrects ? Justifier votre réponse ?

|                                                                                                    |                                                                                                                          |                                                                                                                                |
|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 1. Exemple 1:<br><br>int &f();<br>float x;<br>x = f();                                             | 2. Exemple 2:<br><br>int n = 3;<br>float x = 3.5;<br>int &f(...)<br>{<br>...<br>return 5;<br>return n;<br>return x;<br>} | 3. Exemple 3:<br><br>int n = 3;<br>float x = 3.5;<br>const int &f(...)<br>{<br>...<br>return 5;<br>return n;<br>return x;<br>} |
| 4. Exemple 4:<br><br>const int &f(...);<br>int n;<br>float x;<br>...<br>f() = 2*n + 5;<br>f() = x; |                                                                                                                          |                                                                                                                                |

**Exercice 5 :** Ecrire une fonction float puissance(float x,int n) qui renvoie  $x^n$  . La mettre en œuvre en utilisant les propriétés de conversion de type, puis les propriétés des arguments par défaut. Par défaut, la fonction puissance devra fournir  $x^4$ .

**Exercice 6 :** Ecrire la macro-fonction qui calcule factoriel de n. Puis la fonction inline qui fait de même. Montrer et expliquer sur un exemple que l'appel n'est pas réalisé correctement avec la macro fonction alors qu'il l'est avec la fonction.

**Exercice 7 :** Ecrire une fonction void affiche(float x, int n = 0) qui affiche  $x^n$  (avec en particulier  $x^0 = 1$  et donc,  $0^0 = 1$ ).

Ecrire une autre fonction void affiche(int n, float x=0) qui affiche  $x^n$  (avec en particulier  $0^n = 1$  et donc,  $0^0 = 1$ ). Les mettre en œuvre dans le programme principal, en utilisant la propriété de surdéfinition.

Remarque : Cet exemple conduit à une erreur de compilation lors d'appel de type mn avec m et n entiers.

**Exercice 8 :**

Qu'il est le résultat fournit par le programme ci-dessous

```
#include <iostream.h>
#include <conio.h>
void essai(float x,char c,int n=0) {
cout<<"Fonction N°1: x = "<<x<<" c = "<<c<<" n = "<<n<<"\n";
}
void essai(float x,int n)
{cout<<"Fonction N°2: x = "<<x<<" n = "<<n<<"\n";}

main() {
char l='z';
int u=4;
float y = 2.0;
essai(y,l,u);
essai(y,l);
essai(y,u);
essai(u,u);
essai(u,l);
essai(y,y);
essai(y,y,u);
getch();}
```

**Exercice 9 :**

Déclarer un tableau de 5 réels. Calculer et afficher leur moyenne. Remarque : - Il ne faut pas utiliser conjointement malloc et delete ou bien new et free.

**Exercice 10 :**

Ecrire un programme allouant des emplacements pour des tableaux d'entiers dont la taille est fournie donnée. Les allocations se poursuivent jusqu'à ce que l'on aboutisse à un débordement mémoire.

L'exécution se présentera ainsi:

Donner la taille du tableau (en octets) = 100000000

Allocation bloc numero : 1

Allocation bloc numero : 2

Allocation bloc numero : 3

Allocation bloc numero : 4

Allocation bloc numero : 5

Allocation bloc numero : 6

Allocation bloc numero : 7

Memoire insuffisante - arret execution

Un autre exemple d'exécution:

Donner la taille du tableau (en octets) = 200000000

Allocation bloc numero : 1

Allocation bloc numero : 2

Allocation bloc numero : 3

Memoire insuffisante - arret execution